Bonus Week Lecture Notes: Vibe Coding in Engineering

AI writes the code, the engineer verifies it: checks, tests, review and limits

AI for Aerospace & Mechanical Engineering (PhD), bonus week

> **Provenance:** new content (bonus week). Definitions, the head-chef idea, the cautionary tales and the delegation rules are paraphrased from G. Kim and S. Yegge, *Vibe Coding* (IT Revolution, 2025), Chs. 1, 4 and 7. Karpathy's coinage and the review rule come from S. Willison, "Not all AI-assisted programming is vibe coding" (19 Mar 2025). The safety angle was cross-checked with N. AlNasser, *The Vibe Coder's Handbook* (2026), Chs. 14 and 17; its unsourced statistics are not used. Code verification follows S. P. Veluri, C. J. Roy and E. A. Luke, *Computers & Fluids* (2012); mutation testing follows R. A. DeMillo, R. J. Lipton and F. G. Sayward, *IEEE Computer* 11(4), 34–41 (1978). Study results are listed under Sources, each read in the original on 2026-10-16. All other numbers were computed in the course environment (lab notebook).

1. The problem

The UAV of project module M3 needs pressure altitude from its barometer. One prompt to an AI assistant ("write `pressure_altitude(p)` for the ISA") can return answers like the four in the lab, written as teaching examples. All four run, carry docstrings and return a number. At 50 kPa they give 5574.4 m, 172.2 m, −159 888.8 and 5574.4 m. Which one would you fly?

Andrej Karpathy coined "vibe coding" in February 2025 for giving in to the vibes and forgetting "that the code even exists", which he found acceptable for "throwaway weekend projects". Simon Willison drew the line that matters for engineers: if you review, test and can explain the code, you are not vibe coding. Kim and Yegge use the term broadly but keep the same rule in their *head chef* image: you may delegate the cooking, never the responsibility for what leaves the kitchen. This week is about that second mode.

2. Why plausible code is dangerous

AI-written code rarely fails by crashing. It fails the way human code fails, only faster and with confident comments: a wrong constant, an undocumented unit interface, a "safety" clamp that changes the physics, a typo that lowers the accuracy of a method. Kim and Yegge (Ch. 4) report an agent that silently disabled tests and deleted 80 % of the cases in one suite, an unreadable 3 000-line function, a repository lost during a branch clean-up, an agent that proposed wiping the storage of a CNC machine, and an agent that ignored an explicit instruction. None of this needs AI: the Mars Climate Orbiter was lost in 1999 because a ground software file ("Small Forces") produced thruster impulse in lbf·s where the navigation software expected N·s (NASA Mishap Investigation Board, Phase I report).

Worked Example 1: the right answer by hand

In the ISA troposphere $T = T_0 - Lh$; hydrostatics and the ideal-gas law give

$$h = \frac{T_0}{L}\left[1 - \left(\frac{p}{p_0}\right)^{RL/g_0}\right], \quad T_0 = 288.15\ \text{K},\ L = 0.0065\ \tfrac{\text{K}}{\text{m}},\ R = 287.05\ \tfrac{\text{J}}{\text{kg K}},\ g_0 = 9.80665\ \tfrac{\text{m}}{\text{s}^2}.$$

At $p = 50\,000$ Pa: exponent $287.05 \times 0.0065/9.80665 = 0.19026$; ratio $50\,000/101\,325 = 0.49346$ and $0.49346^{0.19026} = 0.87425$; $T_0/L = 44\,331$ m; so $h = 44\,331 \times 0.12575 \approx 5575$ m (5574.4 m at full precision). Candidate B used the universal gas constant 8.314 J/(mol K): exponent 0.005511, about 35 times too small, and $h \approx 172$ m. A dimension check of RL/g_0 (it must be dimensionless) finds this in one line.

3. A portfolio of checks

Reference values come from a source independent of the code. The ISA table (ISO 2533:1975) gives 101 325 Pa at 0 m and 22 632 Pa at 11 000 m. The table is rounded to 1 Pa and $|dh/dp| = RT/(pg_0) \approx 0.28$ m/Pa near 11 km, so ±1 m is a justified tolerance. B and C fail; D, which clamps the altitude to 0 m at and above 101 325 Pa "for safety", passes.

Properties must hold for every valid input: altitude decreases with pressure; the round trip pressure → altitude → pressure returns the input; a high-pressure day (105 kPa) gives a negative altitude. They are checked on 200 seeded random samples, so every failure can be reproduced.

Differential testing compares the new code with an independent trusted implementation over the whole operating range (here: the Week 2 library function, 30 to 110 kPa, ±1 m). Candidate A with its rounded constants deviates by at most 0.002 m and passes everything. Candidate C is the textbook formula in hPa and feet: fed pascals it returns nonsense without an error, and with an adapter (Pa → hPa, ft → m) it agrees to 3.88 m. Its defect is a missing unit contract, exactly the Mars Climate Orbiter pattern. Result: A passes all five checks; B and C fail four; D fails four of five although it passed the table.

4. Code verification: the observed order of accuracy

For numerical methods a single tolerance test is weak. Code verification asks whether the discretisation error falls at the rate the method promises. With errors e_1, e_2 at step sizes h_1, h_2, the observed order is (Veluri, Roy and Luke, 2012, Eq. 4)

$$\hat{p} = \frac{\ln(e_1/e_2)}{\ln(h_1/h_2)}.$$

Worked Example 2: an RK4 with a one-character typo

One classical RK4 step for $\dot{x} = -x$, $x_0 = 1$, $h = 0.1$: $k_1 = -1$, $k_2 = -0.95$, $k_3 = -0.9525$, $k_4 = -0.90475$, so $x_1 = 1 + \frac{0.1}{6}(-1 - 1.9 - 1.905 - 0.90475) = 0.9048375$; exact $e^{-0.1} = 0.904837418$, error 8.2×10^{-8}. Now integrate a pitch oscillator $\ddot{\theta} + 2\zeta\omega_n\dot{\theta} + \omega_n^2\theta = 0$ ($\omega_n = 3$ rad/s, $\zeta = 0.5$, $\theta_0 = 0.1$ rad) to $T = 2$ s with a correct RK4 and with a version that computes k_3 from k_1 instead of k_2. At $h = 0.05$ both pass a 10^{-5} tolerance test (errors 1.21×10^{-7} and 6.93×10^{-6}). The observed orders tell the truth:

h (s)	0.1	0.05	0.025	0.0125	observed orders
Euler	7.25e-3	2.78e-3	1.23e-3	5.80e-4	1.38, 1.18, 1.09
RK4 with typo	4.48e-5	6.93e-6	1.19e-6	2.29e-7	2.69, 2.55, 2.38
RK4	1.87e-6	1.21e-7	7.65e-9	4.80e-10	3.95, 3.99, 4.00

By hand from the typo row: $\ln(6.929/1.186)/\ln 2 = 2.547$, not 4. When no exact solution exists, the method of manufactured solutions supplies one.

5. Testing the tests: mutation testing

Tests that pass prove nothing if they would also pass on wrong code. Mutation testing seeds known defects (mutants) into trusted source and counts how many the suite catches (kills). On six mutants of the ISA function (universal R, hPa reference, sea-level clamp, inverted exponent, sign error, scalars only) four suites that all pass on correct code score: weak ("runs, 0 m at sea level") 1/6; typical (+ one hand value) 4/6; strong (+ table, below sea level, array, monotonic) 6/6; and the strong suite with the assertions that D fails deleted, the "make the tests pass" move of the cautionary tale, falls back to 4/6. Tracking the score turns a silent deletion into a visible drop.

6. Static checks and the environment

Before running anything, count what you asked for (Kim and Yegge's "count your babies"): an answer may miss a requested function or add unrequested ones. Then list imports that do not resolve. Spracklen et al. (USENIX Security 2025) generated 576 000 samples with 16 models: on average at least 5.2 % of the packages recommended by commercial models and 21.7 % by open-source models did not exist, with 205 474 unique invented names that an attacker could register. Finally, pin the environment: in the course environment (NumPy 1.26.4) `np.trapz` exists and `np.trapezoid` does not; in NumPy 2.5.3 it is the reverse. Holzmann's "Power of 10" rules for safety-critical code transfer well: at least two assertions per function (rule 5) and zero warnings with daily static analysis (rule 10).

7. What the evidence says, and its limits

In a randomised trial (Peng et al., 2023), 95 freelancers wrote an HTTP server in JavaScript 55.8 % faster with GitHub Copilot (95 % CI 21–89 %). In another (METR, 2025), 16 experienced developers working on 246 real issues in their own large repositories took **19 % longer** with early-2025 tools (CI +2 to +39 %), although they had forecast a 24 % speed-up and afterwards believed in 20 %. METR's 2026 follow-up measured −18 % (CI −38 to +9 %) and −4 % (CI −15 to +9 %) and called it very weak evidence. The DORA 2024 survey estimated that as AI adoption increased, delivery throughput fell 1.5 % and stability 7.2 %. On security, about 40 % of 1 689 Copilot programs in 89 risky scenarios were vulnerable (Pearce et al.), and users with an assistant wrote less secure code while believing it more secure (Perry et al.). A vendor test (Veracode, 2025) reported flaws in 45 % of cases; peer review is not stated. Conclusion: large gains on small, well-specified tasks are real; in complex,

familiar code they may vanish, and self-reported speed-ups are not evidence. Several popular statistics (for example “63 % of users are non-developers”) have no traceable source and are not used here.

Worked Example 3: a static-fire log (practice assignment)

A synthetic load-cell log (half-sine, 120 N, 1.6 s; exact impulse $2Ft_b/\pi = 122.23$ N·s) has 498 jittered samples and three dropped packets. Clipping negative noise and applying the trapezoid rule on the actual timestamps gives $I = 122.30$ N·s and, with 65 g propellant, $I_{sp} = I/(mg_0) = 191.86$ s. The common AI shortcut `dt = t[1] - t[0]` gives 97.85 N·s (20 % low). A convergence check on an even grid cannot see this bug; a reference case on an uneven grid can. Every check sees some defects, which is why we combine them.

8. A working method

Delegate small, well-defined pieces with checkable criteria; supervise more when the task is novel or safety-relevant; forbid edits to tests, data and hardware; check after every step and commit before every agent change; keep a project file with units, conventions and pinned versions (after Kim and Yegge, Ch. 4). Write prompts that state units, ranges, environment and two or three reference values, and ask for assumptions and edge cases before code. Never paste data that is not cleared for external services, and install nothing before checking the package and its publisher. You sign the result.

Summary. We can now tell vibe coding from AI-assisted engineering, verify AI code with reference values, properties and differential tests, measure the observed order of numerical code, judge test suites by their mutation score, run static checks, and read AI studies critically.

Sources

Kim, G., Yegge, S. *Vibe Coding*. IT Revolution, 2025 · Willison, S. “Not all AI-assisted programming is vibe coding”, simonwillison.net, 19 Mar 2025 · Peng, S. et al. arXiv:2302.06590 · METR, arXiv:2507.09089 and metr.org blog, 24 Feb 2026 · Google Cloud, DORA 2024 report blog · Pearce, H. et al. arXiv:2108.09293 · Perry, N. et al. ACM CCS 2023, doi:10.1145/3576915.3623157 · Spracklen, J. et al. USENIX Security 2025, arXiv:2406.10279 · Veracode press release, 30 Jul 2025 · NASA MCO Mishap Investigation Board, Phase I Report, 10 Nov 1999 · Holzmann, G. J. “The Power of 10”, NASA/JPL · Veluri, Roy, Luke, *Computers & Fluids*, 2012 · DeMillo, Lipton, Sayward, *IEEE Computer* 11(4), 1978 · ISO 2533:1975 Standard Atmosphere.